18. Design Video Streaming Platform

Key Terms:

Term	Full Name	What it does
HLS	HTTP Live Streaming	Apple's adaptive streaming protocol using .m3u8 playlists
DASH	Dynamic Adaptive Streaming over HTTP	Open standard for adaptive streaming
Transcoding	Format conversion	Converting video to multiple resolutions/codecs
CDN	Content Delivery Network	Edge servers for global video delivery
Bitrate	Data rate	Video quality measure (e.g., 5Mbps for 1080p)
Segment	Video chunk	2-10 second piece of video for streaming
ABR	Adaptive Bitrate	Player switches quality based on bandwidth

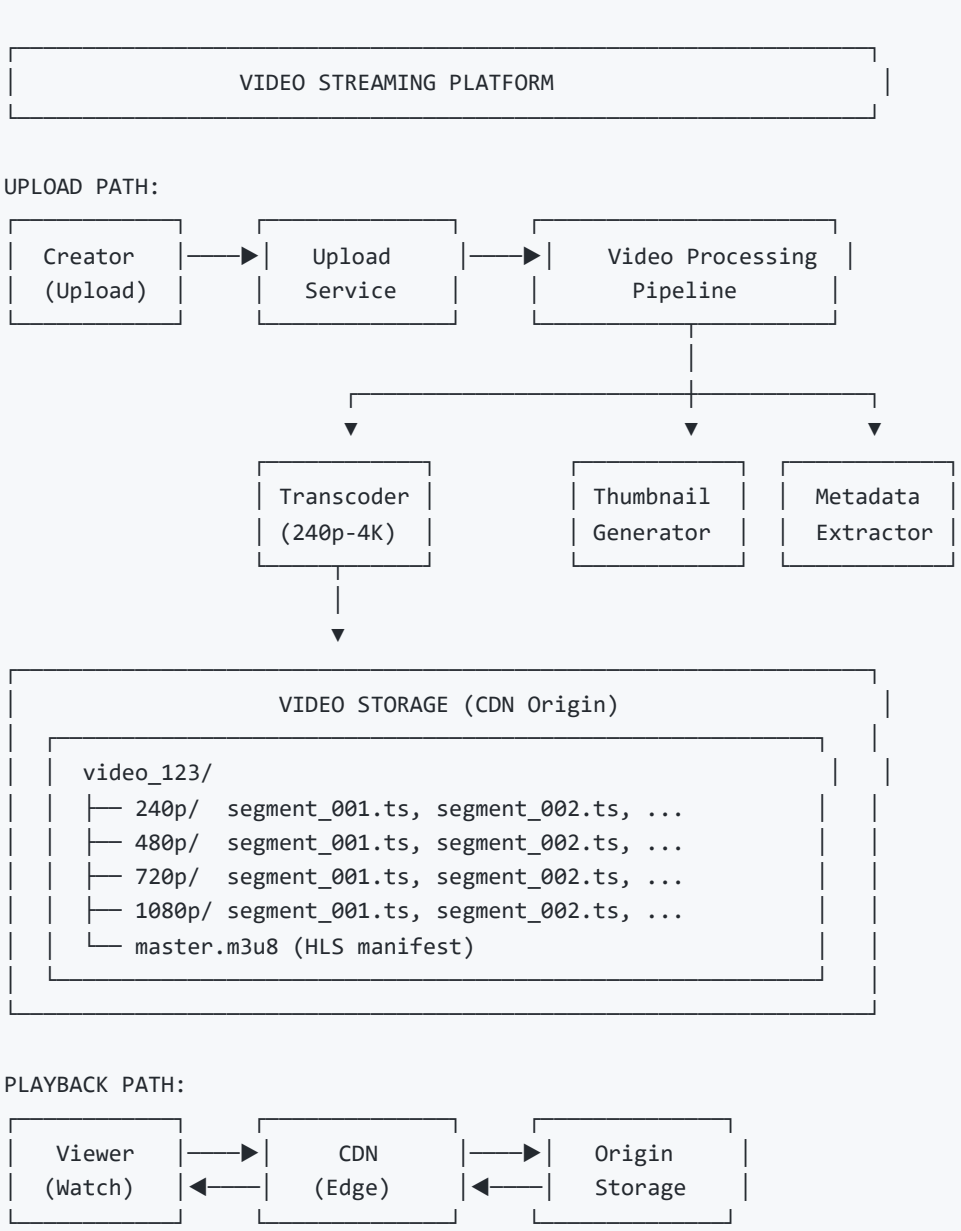
Problem Statement

Design a video streaming platform like YouTube or Netflix that handles video upload, processing, storage, and playback at scale.

Clarifying Questions to Ask

1. **Scale**: How many videos? Daily views? (1B videos, 5B views/day?)
2. **Video length**: Short clips or long movies?
3. **Quality**: Multiple resolutions? Adaptive bitrate?
4. **Live streaming**: Pre-recorded only or live too?
5. **Latency**: How fast should videos be available after upload?
6. **Features**: Comments, likes, recommendations?
7. **Monetization**: Ads, subscriptions, or both?

High-Level Design



API Design

Video APIs

Method	Endpoint	Description
`POST`	`/api/v1/videos/upload`	Initialize video upload
`PUT`	`/api/v1/videos/upload/{uploadId}/chunk`	Upload video chunk
`POST`	`/api/v1/videos/upload/{uploadId}/complete`	Complete upload & start processing
`GET`	`/api/v1/videos/{videoId}`	Get video metadata

`GET`	`/api/v1/videos/{videoId}/stream`	Get streaming manifest (HLS/DASH)
`DELETE`	`/api/v1/videos/{videoId}`	Delete a video

Playback APIs

Method	Endpoint	Description
`POST`	`/api/v1/videos/{videoId}/watch`	Record watch event & get playback token
`PUT`	`/api/v1/videos/{videoId}/progress`	Update watch progress
`GET`	`/api/v1/videos/{videoId}/manifest.m3u8`	HLS manifest

Feed & Discovery APIs

Method	Endpoint	Description
`GET`	`/api/v1/feed?page={n}`	Get personalized video feed
`GET`	`/api/v1/search?q={query}`	Search videos
`GET`	`/api/v1/videos/{videoId}/related`	Get related videos

Interaction APIs

Method	Endpoint	Description
`POST`	`/api/v1/videos/{videoId}/like`	Like a video
`POST`	`/api/v1/videos/{videoId}/comments`	Post a comment
`GET`	`/api/v1/videos/{videoId}/comments?limit=20`	Get comments

Key Components

Component	Purpose	Technology
Upload Service	Handle file uploads	Chunked upload API
Transcoding	Convert to multiple formats	FFmpeg, Media Services
Storage	Store video segments	Blob Storage

CDN	Global video delivery	Azure CDN, Akamai
Metadata DB	Video info, stats	PostgreSQL
Search	Video discovery	Elasticsearch
Recommendation	Personalized suggestions	ML service

Video Processing Pipeline

```
TRANSCODING WORKFLOW:
┌─────────────────────────────────────────────────────────────────┐
│  1. UPLOAD: Chunked upload to temporary storage                 │
│     - Resume support for large files                            │
│     - Validate format, check for duplicates                     │
│                                                                  │
│  2. QUEUE: Add to processing queue (Kafka/Service Bus)          │
│                                                                  │
│  3. TRANSCODE: Convert to multiple resolutions                  │
│     ┌──────────────┐                                            │
│     │ Original     │                                            │
│     │ 1080p 5Mbps  │                                            │
│     └──────┬───────┘                                            │
│            │                                                     │
│     ┌──────┼──────┬──────┬──────┬──────┐                       │
│     ▼      ▼      ▼      ▼      ▼      ▼                       │
│   240p   480p   720p   1080p   4K                               │
│   400k   1.5M   3Mbps  5Mbps   15Mbps                           │
│                                                                  │
│  4. SEGMENT: Split into 2-10 second chunks (HLS/DASH)           │
│                                                                  │
│  5. STORE: Upload to CDN origin                                 │
│                                                                  │
│  6. INDEX: Update metadata, mark as ready                       │
└─────────────────────────────────────────────────────────────────┘
```

Adaptive Bitrate Streaming

```
HLS (HTTP Live Streaming):
┌─────────────────────────────────────────────────────────────────┐
│  master.m3u8 (Master Playlist):                                 │
│  ┌──────────────────────────────────────────────────────┐       │
│  │ #EXTM3U                                              │       │
│  │ #EXT-X-STREAM-INF:BANDWIDTH=400000,RESOLUTION=426x240    │       │
│  │ 240p/playlist.m3u8                                   │       │
│  │ #EXT-X-STREAM-INF:BANDWIDTH=1500000,RESOLUTION=854x480   │       │
│  │ 480p/playlist.m3u8                                   │       │
│  │ #EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1280x720  │       │
│  │ 720p/playlist.m3u8                                   │       │
```

```
│  └──────────────────────────────────────────────────────────┘    │
│                                                                  │
│  Player behavior:                                                │
│  1. Fetch master playlist                                        │
│  2. Measure bandwidth (e.g., 2 Mbps)                             │
│  3. Select appropriate quality (480p)                            │
│  4. Fetch segments from 480p playlist                            │
│  5. Continuously adapt based on network conditions               │
└──────────────────────────────────────────────────────────────────┘
```

CDN and Caching

```
CACHE HIERARCHY:
┌──────────────────────────────────────────────────────────────────┐
│  L1: Client buffer (10-30 seconds of video)                      │
│  L2: Edge POP (popular videos, high hit rate)                    │
│  L3: Regional cache (less popular videos)                        │
│  L4: Origin storage (all videos)                                 │
│                                                                  │
│  Cache strategy:                                                 │
│  - Popular videos: Push to all edges                             │
│  - Long tail: Pull on demand, cache at edge                      │
│  - New uploads: Pre-warm cache in creator's region               │
└──────────────────────────────────────────────────────────────────┘
```

Database Schema

```sql
-- Videos
CREATE TABLE videos (
    id BIGINT PRIMARY KEY,
    creator_id BIGINT,
    title VARCHAR(255),
    description TEXT,
    duration_seconds INT,
    upload_status VARCHAR(20),  -- processing, ready, failed
    visibility VARCHAR(20),     -- public, private, unlisted
    view_count BIGINT DEFAULT 0,
    like_count BIGINT DEFAULT 0,
    created_at TIMESTAMP,
    published_at TIMESTAMP
);

-- Video Files (different qualities)
CREATE TABLE video_files (
    video_id BIGINT,
    resolution VARCHAR(10),     -- 240p, 480p, 720p, 1080p
    bitrate INT,
    codec VARCHAR(20),
    storage_path TEXT,
```

```
    file_size BIGINT,
    PRIMARY KEY (video_id, resolution)
);

-- View counts (separate for write scalability)
CREATE TABLE view_counts (
    video_id BIGINT PRIMARY KEY,
    view_count BIGINT,
    last_updated TIMESTAMP
);
```

Scalability Considerations

- **Upload**: Chunked uploads with resume capability
- **Processing**: Parallel transcoding jobs, auto-scale workers
- **Storage**: Tiered storage (hot for new, cold for old)
- **CDN**: 95%+ cache hit ratio target
- **View counts**: Async aggregation to avoid write contention

Trade-offs

Decision	Option A	Option B	Recommendation
Streaming	Progressive	Adaptive (HLS)	**HLS** - better UX
Transcoding	On-upload	On-demand	**On-upload** - faster playback
Storage	Single quality	All qualities	**All** - adaptive streaming
View counts	Real-time	Batched	**Batched** - scale

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
CDN	Global video segment delivery with edge caching	Tech Guide
Kafka	Video processing job queue and event distribution	Tech Guide
Redis	View count aggregation and metadata caching	Tech Guide

How to Explain It Well

> **Opening**: "Video streaming needs efficient processing and global delivery. I'll design for 5 billion daily views with adaptive bitrate streaming."

> **Key talking points**:
>
> - "Transcode to multiple resolutions: 240p to 4K for all devices"
> - "HLS segments allow adaptive bitrate - switch quality mid-stream"
> - "CDN serves 95%+ of requests from edge cache"
> - "Async view count aggregation to avoid database bottleneck"
> - "Chunked upload with resume for large file reliability"
